

DOCUMENTATION

Telegram Bot RTK Base Monitor

Real-time monitoring system for GNSS RTK base station
Raspberry Pi + rtkbase + Telegram Bot

Version 5.0 - June 2026

Author: Rocco Carpi

Table of Contents

1.	System Overview	3
2.	Typical Architecture	3
3.	Prerequisites	4
4.	Creating the Telegram Bot	4
5.	Script Configuration	5
6.	Installing the Script	6
7.	Configuring the systemd Service	7
8.	Bot Commands	8
9.	Automatic Alerts	8
10.	Script Structure	9
11.	Maintenance and Updates	10
12.	Troubleshooting	10
13.	File and Variable Reference	11

1. System Overview

The RTK Base Monitor is a lightweight bash script that runs in the background on a Raspberry Pi and monitors the status of a GNSS RTK base station in real time. The system sends automatic notifications via a Telegram bot in case of disconnections or hardware anomalies, and responds to interactive commands to provide a complete system status report.

Key features:

- Automatic monitoring every 60 seconds (configurable)
- Responds to Telegram commands in under 3 seconds via long polling
- Real-time alarm and reconnection notifications
- Full hardware monitoring: CPU, RAM, temperature, disk, swap
- Disconnection counter with automatic hourly reset
- Extremely lightweight: less than 2MB RAM, near-zero CPU usage
- Automatic startup at boot via systemd service
- No external dependencies - only bash and curl (pre-installed)

2. Typical Architecture

This script is designed for a typical RTK infrastructure consisting of a GNSS base station on a Raspberry Pi that sends corrections to an NTRIP caster (e.g. SNIP) and/or to a public NTRIP network (e.g. Centipede RTK). Adapt the IP addresses and parameters to your specific setup.

Component	Description	Script Variable
Raspberry Pi	GNSS base station with rtkbase str2str, gpsd	- (local machine)
NTRIP Server	e.g. SNIP, RTKLIB caster Receives corrections from base	SERVER_NTRIP_IP
Public NTRIP Network	e.g. Centipede RTK, RTK2go Correction relay	CENTIPEDE_IP
Router	Port forwarding to NTRIP server Only NTRIP port open	-
Dynamic DNS	e.g. DuckDNS, No-IP Allows rovers to connect	-

Typical data flow:

GNSS Receiver -> str2str -> NTRIP Server (LAN) -> Rover via internet
str2str -> Public NTRIP network (e.g. Centipede) [optional relay]

3. Prerequisites

On the Raspberry Pi:

- Raspberry Pi (any model running Raspberry Pi OS)
- rtkbase installed and running (Stefal project on GitHub)
- curl installed (included by default on Raspberry Pi OS)
- SSH access from the local network
- Active internet connection (required for Telegram notifications)

Telegram account:

- Telegram app installed on smartphone or PC
- Active Telegram account

The script does not require Python, Node.js, or any other dependencies. It uses only bash and curl, both already available on any Raspberry Pi OS installation.

4. Creating the Telegram Bot

4.1 Create the bot with BotFather

BotFather is Telegram's official bot for creating and managing other bots.

1. Open Telegram and search for **@BotFather**
2. Start the chat by pressing **Start**
3. Send the command **/newbot**
4. Enter a **name** for your bot (e.g. My RTK Base Monitor)
5. Enter a unique **username** ending with 'bot' (e.g. myrtk_bot)
6. BotFather will reply with your **TOKEN** - save it immediately!

The token looks like this:

```
123456789:ABCdefGHIjklMNOpqrsTUVwxyz
```

WARNING: The token is a private credential. Never share it publicly and never include it in shared files or public repositories.

4.2 Get your Chat ID

The Chat ID identifies the conversation where the bot will send messages. Every Telegram user has a unique Chat ID.

1. Search for your bot on Telegram and press **Start**
2. Open a browser and go to this URL (replace TOKEN with yours):

```
https://api.telegram.org/botTOKEN/getUpdates
```

3. In the JSON response, find the **"id"** field inside **"chat"**
4. That integer number is your Chat ID - save it!

Example JSON response:

```
{"ok":true,"result":[{"update_id":161121724,"message":{"chat":{"id":123456789,"type":"private"}}}]}
```

The Chat ID in this example is 123456789 - yours will be different.

4.3 Set up bot commands in the menu

To display commands in the bot menu on Telegram:

1. Write to @BotFather: **/setcommands**
2. Select your bot from the list
3. Paste exactly this text:

```
status - Full system status and hardware  
uptime - Quick uptime and server connection  
log - Last lines of rtkbase log  
help - List of available commands
```

4.4 Optional customization

- **/setdescription** - Add a description to the bot
- **/setuserpic** - Set the bot profile picture
- **/setname** - Change the bot display name

5. Script Configuration

Before installing the script you need to adapt the configuration variables to your infrastructure. Open the file `monitor_rtk.sh` with a text editor and modify the initial section:

```
TOKEN='YOUR_TOKEN'           # Token obtained from BotFather
CHAT_ID='YOUR_CHAT_ID'       # Chat ID obtained from getUpdates
SERVER_NTRIP_IP='192.168.1.X' # IP of your NTRIP server on LAN
CENTIPEDE_IP='66.185.123.66' # Public NTRIP network IP (or remove)
INTERVAL=60                  # Seconds between each automatic check
```

Variable	Description	Example
TOKEN	Bot token obtained from BotFather	123456789:ABCdef...
CHAT_ID	Your personal Telegram Chat ID	987654321
SERVER_NTRIP_IP	IP of the NTRIP server on your LAN (e.g. PC running SNIP, RTKLIB caster)	192.168.1.10
CENTIPEDE_IP	IP of the public NTRIP network Remove if not used	66.185.123.66
INTERVAL	Seconds between automatic checks Minimum recommended: 30	60

If you do not use a public NTRIP network such as Centipede, you can remove or comment out the lines related to `check_centipede()` and its notifications in the script.

6. Installing the Script

6.1 Connect to the Raspberry Pi via SSH

From a PC on the same local network:

```
ssh USER@RASPBERRY_IP
# e.g.: ssh pi@192.168.1.72
```

6.2 Copy the script to the Raspberry Pi

Method 1 - Copy from PC via SCP (Windows):

```
scp monitor_rtk.sh USER@RASPBERRY_IP:/home/USER/monitor_rtk.sh
```

Method 2 - Create directly on the Raspberry Pi with nano:

```
nano /home/USER/monitor_rtk.sh
# Paste the script content
# Save with Ctrl+X -> Y -> Enter
```

6.3 Insert the token into the file

Quick method via sed (replace YOUR_TOKEN with your actual token):

```
sed -i 's/INSERT_TOKEN_HERE/YOUR_TOKEN/' /home/USER/monitor_rtk.sh
```

Or open it with nano and manually edit the TOKEN line:

```
nano /home/USER/monitor_rtk.sh
```

6.4 Make the script executable

```
chmod +x /home/USER/monitor_rtk.sh
```

6.5 Test the script

Before installing as a service, test it manually:

```
bash /home/USER/monitor_rtk.sh
```

If everything works, you will immediately receive a Telegram startup message. Press Ctrl+C to stop the test and proceed with the service installation.

7. Configuring the systemd Service

To automatically start the bot every time the Raspberry Pi boots, we install the script as a systemd service. Replace USER with your Raspberry Pi username (e.g. pi, basegnss, etc.).

7.1 Create the service file

```
sudo nano /etc/systemd/system/monitor-rtk.service
```

Paste this content (replace USER with your username):

```
[Unit]
Description=RTK Base Monitor Telegram Bot
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=USER
ExecStart=/bin/bash /home/USER/monitor_rtk.sh
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Save with Ctrl+X -> Y -> Enter

7.2 Enable and start the service

```
sudo systemctl daemon-reload
sudo systemctl enable monitor-rtk
sudo systemctl start monitor-rtk
```

7.3 Verify the service status

```
sudo systemctl status monitor-rtk
```

Expected output:

```
Active: active (running) since ...
Main PID: XXXXX (bash)
■■■■■■ sleep 60
```

The 'sleep 60' line means the script is waiting for the next monitoring cycle - this is correct behavior.

7.4 Useful service management commands

Command	Description
sudo systemctl start monitor-rtk	Start the service
sudo systemctl stop monitor-rtk	Stop the service
sudo systemctl restart monitor-rtk	Restart the service

<code>sudo systemctl status monitor-rtk</code>	Check service status
<code>sudo journalctl -u monitor-rtk -f</code>	View logs in real time
<code>sudo journalctl -u monitor-rtk -n 50</code>	Last 50 lines of log
<code>sudo systemctl disable monitor-rtk</code>	Disable automatic startup

8. Bot Commands

Command	Description	Response
/status	Full system status	Connections + Raspberry hardware
/uptime	Quick uptime	Uptime + NTRIP server connection time
/log	Last rtkbase log lines	Last 5 lines of the log
/help	Command list and alerts	All available commands

Example /status response:

```
RTK Base Status
15/06/2026 10:30:00

Connections:
[OK] Internet
[OK] NTRIP Server (192.168.1.X:2101)
[OK] Public NTRIP Network
[OK] str2str
[OK] gpsd

Server connection stability:
Connected since: 3h 45m

Disconnections last hour:
Internet: 0x | Server: 0x | Network: 0x

Raspberry Pi Hardware:
Temperature: 52 C
CPU: 1.2%
Load: 0.15, 0.12, 0.10
RAM: Used: 180MB / 907MB (free: 200MB) - 20%
Swap: Used: 50MB / 199MB - 25%
Disk: Used: 4.0G / 14G (29%)
Uptime: 3 weeks, 2 days, 5 hours
LAN IP: 192.168.1.72
```

9. Automatic Alerts

The bot monitors automatically every 60 seconds. Notifications are sent only when the status changes (from OK to ALARM or vice versa), avoiding repeated messages. For every problem, a RESOLVED notification is also sent when things return to normal.

Alert	Trigger Condition	Configurable
Internet lost	Ping to 8.8.8.8 failed	No (connectivity)

NTRIP server disconnected	TCP connection missing	No (connectivity)
Public NTRIP network disconnected	TCP connection missing	No (connectivity)
str2str stopped	Process not found (pgrep)	No (process)
gpsd stopped	Process not found (pgrep)	No (process)
High CPU temperature	Temperature > 75 degrees C	Yes - temp_raw variable
High RAM usage	RAM used > 80%	Yes - ram_pct variable
Swap almost full	Swap used > 90%	Yes - swap_pct variable
Disk almost full	Disk used > 80%	Yes - disco_pct variable

10. Script Structure

10.1 Configuration variables

```
TOKEN='...'          # Telegram bot token
CHAT_ID='...'        # Recipient Chat ID
SERVER_NTRIP_IP='...' # NTRIP server IP
CENTIPEDE_IP='...'   # Public NTRIP network IP
INTERVAL=60          # Seconds between each check
```

10.2 Connection check functions

```
check_internet()      # ping -c1 -W3 8.8.8.8
check_server()        # ss -tnp | grep IP:2101
check_centipede()     # ss -tnp | grep IP:2101
check_str2str()       # pgrep -x str2str
check_gpsd()          # pgrep -x gpsd
```

10.3 Hardware functions

```
get_temperatura()     # /sys/class/thermal/thermal_zone0/temp
get_cpu()              # top -bn1 snapshot
get_ram()              # free -m
get_ram_pct()          # RAM usage percentage
get_swap()             # free -m swap
get_swap_pct()         # Swap usage percentage
get_disco()            # df -h /
get_disco_pct()        # Disk usage percentage
get_uptime()           # uptime -p
get_carico()           # Load average
get_ip()               # hostname -I
```

10.4 Main loop

The loop continuously performs these operations:

1. Telegram long polling with 2-second timeout - if a command arrives it executes immediately
2. Checks if INTERVAL seconds have passed since last check - if so, runs automatic monitoring
3. In monitoring: compares current state with previous state, sends notification only if changed
4. Resets disconnection counters every 60 minutes

Long polling is the technique that enables fast command responses without stressing the Raspberry Pi: the script waits for a response from Telegram for 2 seconds, then resumes the cycle. CPU consumption is practically zero.

10.5 Disconnection counter

For each type of disconnection, a counter is incremented. Counters are automatically reset every hour and are visible in the /status command under the 'Disconnections last hour' section.

10.6 Server connection uptime

The NTRIP server connection timestamp is saved at startup and updated on every reconnection. The /status command shows how long the connection has been active in hours and minutes.

11. Maintenance and Updates

11.1 Updating the script

1. Stop the service:

```
sudo systemctl stop monitor-rtk
```

2. Replace the file with the new version:

```
nano /home/USER/monitor_rtk.sh
```

3. Re-insert the token if needed:

```
sed -i 's/INSERT_TOKEN_HERE/YOUR_TOKEN/' /home/USER/monitor_rtk.sh
```

4. Restart the service:

```
sudo systemctl start monitor-rtk
```

11.2 Modifying alert thresholds

Thresholds are in the monitoraggio() function of the script:

```
[ "$temp_raw" -gt 75000 ] # 75000 = 75 degrees C (raw x1000)
[ "$ram_pct" -gt 80 ]    # 80 = 80% RAM
[ "$swap_pct" -gt 90 ]   # 90 = 90% Swap
[ "$disco_pct" -gt 80 ]  # 80 = 80% Disk
```

11.3 Modifying the monitoring interval

```
INTERVAL=60 # Change the value in seconds (minimum recommended: 30)
```

11.4 Backup the script

```
cp /home/USER/monitor_rtk.sh /home/USER/monitor_rtk_backup.sh
```

12. Troubleshooting

Problem	Likely Cause	Solution
Bot does not respond to commands	Service not running	sudo systemctl start monitor-rtk
Telegram message not received	Wrong or expired token	Check TOKEN in the script file
getUpdates returns []	Bot not started on Telegram	Press Start in the bot chat
Service keeps restarting	Script error	journalctl -u monitor-rtk -n 50
Chat ID not found in getUpdates	You haven't messaged the bot	Send /start to the bot and retry
/log shows nothing	Different log path	Edit get_log() with correct path

Double %% in messages	Bash character escaping	Use %25 instead of %% in strings
Temperature alert always firing	Threshold too low	Increase 75000 in monitoraggio()
str2str always shown as stopped	Different process name	Check with: pgrep -a str2str

13. File and Variable Reference

Path	Description
/home/USER/monitor_rtk.sh	Main bot script
/etc/systemd/system/monitor-rtk.service	systemd service file
/sys/class/thermal/thermal_zone0/temp	CPU temperature (raw, divide by 1000)
/home/USER/rtkbase/log/str2str.log	rtkbase log (typical path, may vary)

Configurable script variables

Variable	Description	Must Change
TOKEN	Telegram bot token from BotFather	YES - required
CHAT_ID	Your personal Telegram Chat ID	YES - required
SERVER_NTRIP_IP	IP of the NTRIP server on LAN	YES - required
CENTIPEDE_IP	Public NTRIP network IP	YES or remove if not used
INTERVAL	Seconds between automatic checks	Optional (default 60)